



# Tecnológico de Monterrey

Sprint 2 (Advanced Web) (Francisco)

## **Equipo #21**

Santos Alejandro Arellano Olarte // **A01643742**

Roberto Anwar Teigeiro Aguilar // **A01643651**

Fernando Antonio Lopez Garcia // **A01643685**

Diego Michell Villa Duran // **A00836723**

Sadrac Aramburo Enciso // **A01643639**

28/Abril/2025

## Enlace al Repositorio GitHub

 URL del Repositorio: <https://github.com/Santos-Arellano/Software-System-Planning>

Rama principal: *main* → Carpeta */Web\_francisco/Sprint 2\_(Advanced Web)/sistema-tareas-ti*

## 1. Roles y Niveles de Trabajadores

### Backend (Spring Boot)

- **Entidades:**

java

```
// User.java
@Enumerated(EnumType.STRING)
private Role role; // PROGRAMADOR
@Enumerated(EnumType.STRING)
private Level level; // JUNIOR/SENIOR
```

- 
- java

```
// Task.java
@Enumerated(EnumType.STRING)
private TaskStatus status; // PENDING, ASSIGNED, COMPLETED
private boolean critical; // Prioridad
```

- 
- **Base de datos:**

- H2 en memoria con datos iniciales en *data.sql*:

sql

```
INSERT INTO users (name, role, level) VALUES
('Juan Pérez', 'PROGRAMADOR', 'SENIOR'),
('Ana López', 'PROGRAMADOR', 'JUNIOR');
```

-

- **Endpoints REST:**

- **GET /api/users:** Lista todos los trabajadores.
- **POST /api/tasks:** Crea nuevas tareas.

## Frontend (React)

- **Selección de usuario:**

tsx

```
// App.tsx
<Select value={currentUser?.id} onChange={handleUserChange}>
  {users.map(user => (
    <MenuItem key={user.id} value={user.id}>
      {user.name} ({user.level})
    </MenuItem>
  ))}
</Select>
```

- 

- **Dashboards diferenciados:**

- **Líder:** **LeaderDashboard.tsx** con lista de tareas pendientes y botón de ejecución del agente.
- **Trabajador:** **WorkerDashboard.tsx** con tareas asignadas y opción de marcarlas como completadas.

## 2. Asignación de Tareas en Tiempo Real

### Backend (WebSocket)

- **Configuración:**

java

```
// WebSocketConfig.java
@Override
public void configureMessageBroker(MessageBrokerRegistry config) {
    config.enableSimpleBroker("/topic");
}
```

- 
- **Notificaciones:**

java

```
// WebSocketController.java
@MessageMapping("/requestInitialData")
public void sendInitialData() {
    messagingTemplate.convertAndSend("/topic/tasks",
    taskService.getAllTasksDto());
}
```

- 

## Frontend (Conexión WebSocket)

- **Suscripciones:**

typescript

```
// AppContext.tsx
useEffect(() => {
    const unsubscribeTasks = subscribe('tasks', (tasks) =>
    setTasks(tasks));
    return () => unsubscribeTasks();
}, []);
```

- 

## 3. Agente de Asignación Automática

### Reglas Implementadas

## 1. Disponibilidad:

java

```
// AgentService.java
List<User> availableUsers = userRepository.findByAvailableTrue();
```

2.

## 3. Prioridad a Seniors:

java

```
if (task.isCritical()) {
    List<User> seniors = userRepository.findByLevel(Level.SENIOR);
}
```

4.

## 5. Límite de 4 tareas:

java

```
@Query("SELECT u FROM User u WHERE SIZE(u.tasks) < 4")
List<User> findAvailableUsersWithLessThan4Tasks();
```

6.

### • Ejecución desde Frontend:

tsx

```
// LeaderDashboard.tsx
<Button onClick={runAgent}>Ejecutar Agente</Button>
```

•

## 4. Base de Datos

| Entidad | Campos                        | Relación          |
|---------|-------------------------------|-------------------|
| User    | id, name, role, level, tasks  | @OneToMany → Task |
| Task    | id, title, status, assignedTo | @ManyToOne → User |

## Datos iniciales:

sql

```
INSERT INTO tasks (title, critical) VALUES  
( 'Corregir bug en login', true),  
( 'Actualizar documentación', false);
```

## 5. Estructura del Repositorio

```
backend/  
├── src/  
│   ├── main/  
│   │   ├── java/  
│   │   │   ├── com/  
│   │   │   │   ├── taskmanager/  
│   │   │   │   │   ├── controller/    # Controladores REST  
│   │   │   │   │   ├── model/        # Entidades  
│   │   │   │   │   ├── repository/    # Interfaces para bases de datos  
│   │   │   │   │   ├── service/      # Servicios  
│   │   │   │   │   ├── agent/        # Agente basado en reglas  
│   │   │   │   │   ├── config/       # Configuraciones  
│   │   │   │   │   ├── exception/    # Excepciones personalizadas  
│   │   │   │   │   ├── dto/         # Objetos de transferencia de datos  
│   │   │   │   │   └── TaskManagerApplication.java # Clase principal  
│   │   └── resources/  
│   │       ├── application.properties # Configuración de la aplicación  
│   │       └── data.sql               # Datos iniciales  
│   └── test/                        # Pruebas unitarias  
├── pom.xml                          # Dependencias Maven  
└── README.md
```

```
frontend/  
├── public/  
│   ├── favicon.ico  
│   └── index.html  
├── src/  
│   ├── api/          # Servicios API  
│   ├── components/   # Componentes reutilizables  
│   ├── context/      # Contextos de React  
│   └── hooks/        # Custom hooks
```

```
| |— models/      # Interfaces y tipos
| |— pages/       # Componentes de página
| |— utils/       # Utilidades
| |— App.tsx      # Componente principal
| |— index.tsx    # Punto de entrada
| |— websocket.ts # Configuración WebSocket
|— package.json
|— tsconfig.json
|— README.md
```

## Tecnologías Utilizadas

| Área     | Tecnologías                              |
|----------|------------------------------------------|
| Frontend | React 18, TypeScript 5, Material-UI 5    |
| Backend  | Spring Boot 3.4.5, H2, Lombok, WebSocket |
| DevOps   | Maven, GitHub Actions                    |

## Instrucciones de Despliegue

### 1. Backend:

```
bash
```

```
cd backend
mvn spring-boot:run
# Verificar: http://localhost:8080/h2-console
```

### 2.

### 3. Frontend:

```
bash
```

```
cd frontend
npm install && npm start
# Acceder: http://localhost:3000
```

4.

5. **Credenciales de prueba:**

- **Líder:** Usuario "Juan Pérez" (Senior).
- **Trabajador:** Usuario "Ana López" (Junior).